Rafid: Running a 397B MoE Model on a Laptop with Expert Streaming and Cache-Aware Routing

Zaid A. Lateef A. Kareem
Independent researcher

November 2026

Abstract

Rafid runs Qwen3.5-397B-A17B (115 GB at 2-bit) on a laptop with 31 GB of RAM and an 8 GB GPU at 4.8–6.0 tokens/s. In matched benchmarks on the same machine it writes 1.5–2.0× faster and reads prompts 6–7× faster than llama.cpp with memory-mapped experts, while common desktop apps refuse to load the model at all.

The system streams experts from NVMe into a 21.5 GB RAM cache managed by a decayed-frequency policy, and prefetches the next layer's experts by applying that layer's router to the current hidden state. It reads the prompt while the user is still typing.

Optionally, cache-aware routing replaces a low-ranked expert that is not in RAM with a cached expert the router weighs nearly as much. On 10,840 teacher-forced tokens from 84 multi-turn conversations, the strongest setting keeps the exact model's top choice on 98.9–99.0% of tokens in two runs (mean KL 0.0052–0.0053 nats, perplexity +0.29–0.32%); exact routing reproduces bit for bit. It cuts disk traffic per token by 30% and raises generation speed by 26%.

On the same 84 conversational questions, the exact model answers 80 correctly and the routed modes answer 80–83.

1 Introduction

Mixture-of-experts (MoE) models put frontier quality within reach of a laptop's compute, but not its memory. Qwen3.5-397B-A17B activates about 17B of its 397B parameters per token. Yet even at 2 bits its weights take 115 GB, almost four times the RAM of a well-equipped laptop.

The experts are 108 GB of the 115 GB. The rest (attention, shared weights, routers, embeddings) is 7.2 GB and fits in memory. Each token uses 10 of 512 experts in each of 60 layers, so at any moment most expert weights are idle. This suggests serving experts from a fast NVMe drive and keeping only a working set in RAM.

General-purpose runtimes handle this poorly. Desktop apps check memory before loading and refuse a model larger than RAM. llama.cpp can run it by memory-mapping the file and letting the operating system page experts in. On our laptop this gives 3.1–3.7 tokens/s for writing and 1.5–4.9 tokens/s for reading a prompt of 128–512 tokens.

Rafid is a C/C++ inference platform built on llama.cpp that treats the expert store as a first-class cache. Our contributions:

1. **An expert store for consumer hardware.** Experts are streamed from NVMe into a fixed RAM cache with a decayed-frequency (LFU) policy. Prediction-driven prefetch applies the next layer's router to the current hidden state. With exact routing this alone writes 1.5–1.7× and reads prompts 6–7× faster than llama.cpp's mmap baseline.
2. **Reading the prompt while the user types.** The prompt is processed incrementally from the draft in the text box, and speculatively read in full during pauses. When the user presses send, the answer often starts at once.

3. **Graded cache-aware routing for streaming.** Three settings (light, medium, strong) let a low-ranked expert that is missing from RAM be replaced by a cached expert with a bounded loss of router weight. This raises speed by 13–26% and lowers disk reads per token by 17–30%.

4. **A token-level fidelity test for routing changes.** We read 10,840 tokens of the exact model's own answers back under each routing mode, inside their real multi-turn conversations. Per token we measure agreement with the exact model's top choice and the KL divergence from it. This is far more sensitive than counting correct answers, which cannot separate the modes on 84 questions.

5. **Multi-turn conversational tests in Arabic.** We use 9 conversations and 84 questions with automatic checks: running ledgers, family relations, JSON, a long report, scheduling, instruction rules, code, trip planning and history.

2 Related work

Inference from flash. LLM in a flash (Alizadeh et al., 2023) [1] runs dense models up to twice the size of DRAM. It keeps parameters in flash and moves them in larger, more contiguous chunks guided by a cost model. It reuses recently activated neurons (windowing) and bundles rows and columns. Rafid applies the same idea at the granularity MoE models give for free: whole experts.

Expert offloading. Eliseev and Mazur (2023) [2] run Mixtral-8x7B on consumer GPUs and free Colab by exploiting MoE properties for offloading. HOBBIT (Tang et al., 2024) [3] loads a lower-precision copy of a missing expert. It predicts later layers' experts from gating inputs and mixes recency, frequency and precision in its cache policy; it reports up to 9.9× decode speedup on llama.cpp. These systems offload from host RAM to the GPU. Rafid targets a model four times larger than host RAM, so the slow tier is the NVMe drive.

Cache-aware routing. The closest work is Mixture of Cache-Conditional Experts (Skliar et al., 2024) [4]. It adds a bias to the router logits of cached experts, scaled by a running average of the logit range, and uses the biased logits only to rank experts. On phones running llama.cpp with half the experts cached, it reports more than 2× faster generation than LRU, perplexity +0.1–3% and MMLU/GSM8K within 0.1%. Their models have 60 to 160 experts per layer (Qwen1.5-MoE, DeepSeek-V2-Lite, Phi-3.5-MoE, Mixtral). Rafid differs in four ways:

- **Missing experts only.** It swaps only experts that are neither cached nor already being read, so routing is unchanged whenever RAM already holds the expert.
- **Bounded replacement.** A replacement must keep a set share of the router weight, and the top-ranked expert is never replaced.
- **Scale.** It runs at 512 experts per layer and 10 per token, streamed from NVMe.
- **Measurement.** It measures the effect token by token inside multi-turn conversations, rather than by perplexity alone.

397B on a laptop. Flash-MoE (Dan Woods, March 2026) [5] ran a custom Qwen3.5-397B-A17B on a 48 GB M3 Max MacBook Pro. It streamed experts from SSD at over 5.5 tokens/s with 2-bit experts, and 4.36 tokens/s with 4-bit. It also cut the experts per token from 10 to 4. Rafid keeps all 10 experts by default, runs on a Windows laptop with 31 GB RAM and a discrete 8 GB GPU, and reports both quality and fidelity.

Memory-mapped baseline. llama.cpp [6] maps model files into memory, so a model larger than RAM runs while the operating system pages weights in on demand. Its `-n-cpu-moe` option keeps expert tensors on the CPU side and the rest on the GPU, the usual setup for large MoE models on small GPUs. We use exactly this as our baseline.

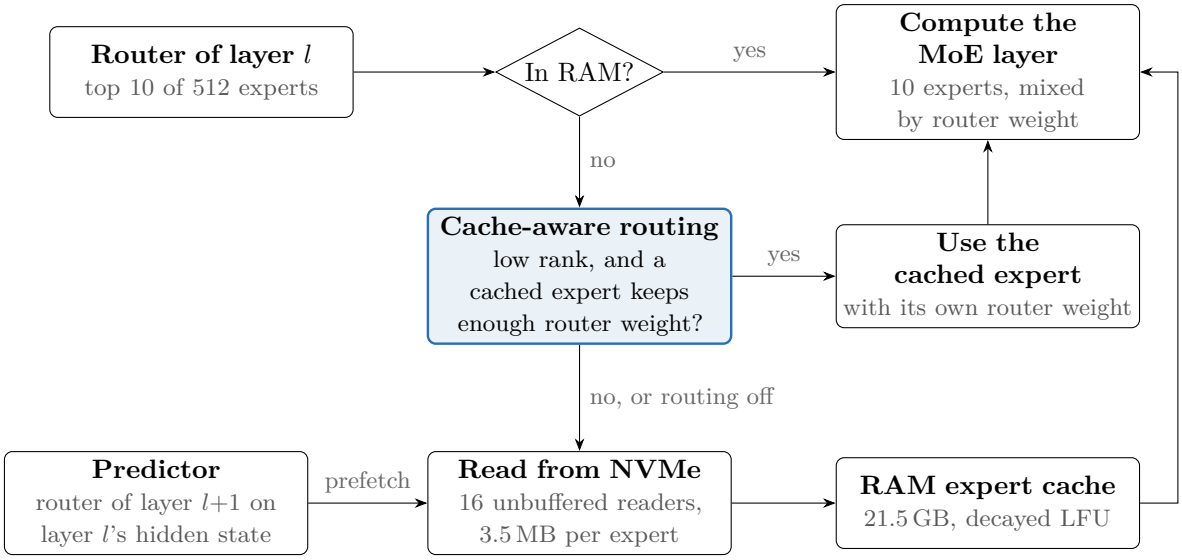


Figure 1: The expert store, per MoE layer while writing a token. Each of the 10 chosen experts takes one of three paths; the predictor reads ahead so that the "in RAM" answer is yes as often as possible. Prompt batches always take the exact path: cache-aware routing acts only when one token is written at a time.

3 System design

Rafid keeps everything except the experts resident, and treats the 108 GB of experts as a cache backed by the NVMe drive. It is about 8,300 lines of C/C++ on top of llama.cpp (b12169), plus a single-page web interface. Figure 1 shows the path each chosen expert takes.

3.1 Expert store

At first load, Rafid splits each GGUF shard into a slim file without expert data, keeping the original shards as the expert source. Expert tensors become slots in a fixed RAM budget; we use 21.5 GB on a 31 GB machine. The MoE layer's routing callback reports the chosen experts. Missing experts are read unbuffered (bypassing the OS page cache) by 16 I/O threads that overlap with computation; one expert is about 3.5 MB at this quantization.

Replacement policy. A decayed least-frequently-used policy ("freq"): each use of an expert adds to a score that halves every 32 tokens, and the lowest score is evicted. An optional "fingerprint" mode pins the experts a profile run used most. With exact routing, the cache serves 82.7% of expert accesses.

Prefetch. While layer l computes, a predictor thread applies the router of layer l+1 to layer l's hidden state, and starts reading the experts it predicts. Wrong guesses are cancelled if not yet started, or sent to the front of the eviction order. Prompt batches use the same predictor on a sample of up to 128 rows.

3.2 Cache-aware routing

During generation (one token per step), a low-ranked expert that is neither in RAM nor already being read can be replaced. The replacement is the cached expert the router weighs most, if its router probability is at least a set share of the original's. The top-ranked expert is never replaced, and an expert already chosen for this token is never chosen twice. Mixing weights are then gathered for the new set, so the replacement contributes with its own weight.

Prompt batches are never routed this way, so the prompt is always read exactly. Table 1 lists the settings, with $k = 10$ experts per token.

Table 1: Cache-aware routing settings.

Mode	Ranks that may be replaced	Minimum weight of the replacement	Expert slots replaced (measured)
off	none	—	0%
light	8–10	70% of the original's	6.5%
medium	6–10	50%	9.6%
strong	4–10	30%	11.6%

The last column is the share of all 6.5 million expert slots in the fidelity test (Section 5.3) that were replaced. Whether an expert is in RAM depends on what the disk delivered by then, so routed runs are not bit-for-bit repeatable; exact routing is.

3.3 Reading the prompt while the user types

Prompt reading is slow when experts stream from disk: every batch touches most of the experts. So Rafid starts before the user presses send.

1. **Read the draft.** As the user types, the page sends the conversation plus the draft. The engine reads it up to the last complete word, minus a two-token margin, in steps of one micro-batch.
2. **Read it as if sent.** When typing pauses for 1.5 s, the full prompt with the generation prefix is read speculatively. If the user then sends that exact text, generation starts with no prompt work left.
3. **Rewind safely.** Qwen3.5 mixes attention with recurrent (Gated DeltaNet) layers, whose state cannot be truncated. Rafid keeps state snapshots at turn ends, question starts and speculative ends, and evicts speculative ones first. Edits to the draft restore the furthest snapshot that still matches.
4. **Yield to the user.** A chat request preempts any reading in progress. The next question can be read as soon as an answer ends.

3.4 Other details

When thinking is switched off, the template's empty think block is kept and the `<think>` and `</think>` tokens are barred from sampling. Without that, the model sometimes reopened a thinking block on its own and spent the whole answer budget in it. While working, Rafid also asks the operating system not to sleep.

4 Experimental setup

All runs use one consumer laptop and one model, with greedy decoding.

Baseline. llama.cpp's own `llama-bench` from the same build Rafid links (b12167 binaries), on the same model files. It uses memory mapping, all layers on the GPU (`-ngl 99`) and all expert tensors on the CPU side (`-ncmoe 99`). We measure writing 64 tokens and reading a 128-token prompt, each twice after a warm-up run, then writing 512 tokens and reading a 512-token prompt once each. Rafid's matched speed test does the same from an empty conversation: a 126- and a 507-token prompt, and exactly 64 and 512 written tokens (the end-of-turn tokens barred), twice each after a warm-up, per routing mode.

Conversation suite. Nine Arabic multi-turn conversations with 84 questions. Each question builds on the model's own earlier answers, as in real use. Every turn has automatic checks, such

Table 2: Hardware, model and decoding.

Item	Value
CPU	Intel Core Ultra 9 275HX (24 threads)
RAM	31 GB; expert cache budget 21.5 GB
GPU	NVIDIA GeForce RTX 5060 Laptop, 8 GB
Disk	NVMe, 6.2 GB/s sequential, 4.2 GB/s random with 16 readers (measured by Rafid)
Model	Qwen3.5-397B-A17B, unsloth UD-IQ2_XXS GGUF, 115 GB; 60 MoE layers, 512 experts, 10 per token
Decoding	temperature 0, no repetition penalty, thinking off, 700 tokens per answer (900–1,200 on long turns)
Context	65,536 tokens

Table 3: The conversation suite.

Conversation	Turns	What carries across turns
Shop ledger	11	running cash balance through purchases and sales
Family	12	relations among people introduced one by one
JSON	9	JSON about Iraqi cities, extended, then queried without JSON
Hospital report	14	a 1,400-token fictional report, then questions on it
Schedule	5	a weekly rota for 5 staff under constraints that change
Rules	9	standing instructions (a signature line, word counts, no commas)
Code	8	reading and changing short Python snippets
Trip	8	distance, time and average-speed arithmetic for one trip
Baghdad	8	dates and ordering of events in the city's history

as a number on the final answer line, required or forbidden words, valid JSON with given fields, a word count, an exact order, or a signature line. Each turn also stores an example answer.

A turn with no answer is not fed back into the conversation; the example answer is used instead, and the turn counts as failed. A first run showed why: one empty answer made the model answer every later question with nothing.

Fidelity test. For each conversation we take the newest run with exact routing in which every turn was answered (10,840 answer tokens, 4,706 prompt tokens). Each mode reads every conversation from scratch. Prompt parts are read in batches, exactly, as a prompt is read. Answer tokens are fed one at a time, as they are written, so routing applies. At each answer position t we compare the routed distribution q_t with the exact one p_t, read first under exact routing:

- **Agreement:** whether argmax q_t equals argmax p_t.
- **KL($\mathbf{p}_t \parallel \mathbf{q}_t$):** over the 32 tokens most likely under p_t plus one bucket for all the rest. This coarsened KL is a lower bound on the full KL, and close to it because those 32 tokens hold nearly all of the mass.
- **Clear disagreements:** positions where that KL exceeds 0.1 nats.
- **Perplexity change** of the exact answer text: exp of the mean difference in negative log-likelihood between q and p, minus one.

Each mode starts from the same cache state: the cache is emptied and refilled from the profile. Speed and disk numbers count answer tokens only.

Table 4: Matched speed tests, tokens/s (speed-up over llama.cpp in brackets). Rafid's numbers are the mean of two runs.

Test	llama.cpp, mmap experts	Rafid, exact	Rafid, light	Rafid, strong
Read a 128-token prompt	1.49	10.70 (7.2×)	10.67	10.94 (7.3×)
Read a 512-token prompt	4.92	28.84 (5.9×)	30.72	32.02 (6.5×)
Write 64 tokens	3.07	5.16 (1.7×)	5.64	6.00 (2.0×)
Write 512 tokens	3.73	5.49 (1.5×)	6.08	6.53 (1.75×)

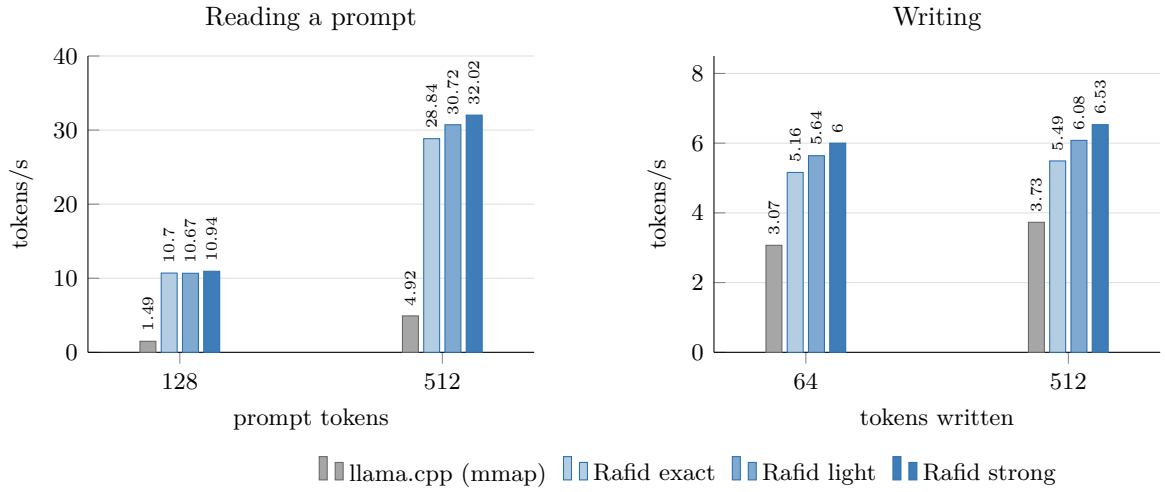


Figure 2: Matched speed tests on the same laptop. Left: reading a prompt; right: writing.

5 Results

5.1 Speed against llama.cpp

In matched runs on the same laptop, Rafid reads prompts 6–7× faster than llama.cpp and writes 1.5–2.0× faster (Table 4, Figure 2). Both tools ran the same tests: a 128- and a 512-token prompt read from an empty context, and exactly 64 and 512 tokens written, twice each after a warm-up (llama.cpp's 512-token runs once).

The gain in reading comes from the expert store: a prompt batch touches most experts of every layer, and Rafid gathers them with large parallel reads and next-layer prediction instead of page faults. llama.cpp's writing speed rises from 3.1 to 3.7 tokens/s over the longer run as the operating system's page cache warms, so the writing gap narrows from 1.7× to 1.5× with exact routing. Routing adds its gain on top: 19% over 512 tokens with strong routing. In the conversation tests of Section 5.2, where answers average 130 tokens, Rafid wrote at 4.8 (exact) to 6.0 (strong) tokens/s.

5.2 Accuracy on multi-turn conversations

All four modes answer 80–83 of the 84 questions (Table 5). No mode is clearly worse than exact routing.

The exact and light numbers combine the first full run with a rerun of the hospital-report conversation. That rerun came after a fix that bars the model from opening a thinking block when thinking is off. The fix changes no other turn, because no other turn opened a thinking block.

The failures are ordinary model errors, not routing artifacts:

Table 5: Conversation suite: 9 Arabic conversations, 84 questions, 107 automatic checks. Speed is the mean writing speed over all answers.

Mode	Questions passed	Checks passed	Speed (tokens/s)
off (exact)	80 / 84	103 / 107	4.77
light	83 / 84	106 / 107	5.39
medium	80 / 84	103 / 107	5.73
strong	83 / 84	106 / 107	6.02

Table 6: Fidelity test: 10,840 answer tokens of the exact model, read back under each routing mode. KL is measured over the reference's 32 likeliest tokens plus one bucket for the rest.

Mode	Same top token	Mean KL (nats/token)	Tokens with KL > 0.1	Perplexity change	Speed (tokens/s)	Disk read per token	Cache hits
off (reference)	100%	0	0%	—	4.61	0.86 GB	82.7%
light	99.20%	0.0036	0.77%	+0.24%	5.25	0.71 GB	87.6%
medium	98.94%	0.0048	1.13%	+0.24%	5.59	0.64 GB	89.6%
strong	98.97%	0.0052	1.22%	+0.29%	5.80	0.60 GB	90.8%

- **Every mode** wrote 6 words when asked for exactly 5.
- **Exact routing** forgot a 600,000-dinar expense in the ledger, confused grandfather and grandmother in the family conversation, and counted 9 letters in a 10-letter word.
- **Medium routing** repeated the family error, ordered two historical events wrongly despite giving the correct years, and ran out of tokens in the rota without a final answer line.

The ranking of the modes (light = strong > off = medium) does not follow routing strength, which points to noise. A three-question difference out of 84 cannot separate the modes. Section 5.3 measures what this test cannot.

5.3 Fidelity of routing

The routed modes keep the exact model's top choice on 98.9–99.2% of tokens (Table 6, first run). Every cost of routing grows with its strength, as the speed gain does.

Each mode read the same 10,840 answer tokens in 84 turns of 9 conversations. Speed here is the speed of reading answers one token at a time, a close proxy for writing them. Strong routing cuts disk traffic per token by 30% and wait time on the disk from 802 s to 355 s over the run.

Repeatability. We ran the test a second time for exact, light and strong routing (Table 7). Exact routing reproduced the first run bit for bit on all 10,840 tokens: the same top choice, summed log-likelihood and agreement with the stored text in every turn. Routed runs depend on what the disk has delivered by each step, so they vary a little, but every measure keeps the modes in the same order. The perplexity change is the least stable measure: light routing moved from +0.24% to +0.15% between runs. Agreement and KL move by far less than the gap between the modes.

Agreement varies little across conversations. JSON is least affected: 99.9–100% for every mode. The Baghdad history conversation is most affected under medium and strong routing (97.9% and 98.1%). The worst single turns still agree on 90–96% of tokens across both runs.

Under exact routing, the model's top token matches the stored answer text on 99.2% of positions, not 100%. Re-tokenizing generated text does not always reproduce the tokens the model wrote. This affects all modes equally, because all compare against the same exact distributions.

Table 7: Two runs of the fidelity test for the routed modes. Exact routing was identical in both.

Mode	Same top token	Mean KL (nats/token)	Tokens with KL > 0.1	Perplexity change	Speed (tokens/s)
light, run 1	99.20%	0.0036	0.77%	+0.24%	5.25
light, run 2	99.06%	0.0037	0.83%	+0.15%	5.14
strong, run 1	98.97%	0.0052	1.22%	+0.29%	5.80
strong, run 2	98.93%	0.0053	1.34%	+0.32%	5.67

6 Discussion and limitations

The expert store gives most of the gain; routing adds a smaller, tunable part. Over 512 written tokens, exact routing already turns llama.cpp's 3.7 tokens/s into 5.5, and strong routing adds another 19%; prompts are read 6–7× faster. A user who wants the exact model loses nothing by using Rafid.

Accuracy tests cannot rank routing modes at this scale. On 84 questions, the four modes land within three answers of each other, in an order unrelated to routing strength. The fidelity test does rank them, consistently on every measure and in both runs. We suggest that work on approximate routing report token-level fidelity alongside task scores.

One token in a hundred. Strong routing changes the exact model's top choice on about 1 token in 100, and clearly shifts its distribution on 1.2%. In free generation, the first changed token sends the text on a different path. So outputs differ more than the per-token number suggests. Of the 84 answers, 37 under light routing were word-for-word identical to the exact ones, 27 under medium and 23 under strong. Whether those different paths are worse is what accuracy tests measure, and there we saw no loss.

Measurements still missing.

- **Standard benchmarks.** Our questions are Arabic and conversational. Perplexity on a standard corpus and a GSM8K or MMLU subset with free-form reasoning would allow comparison with prior work. Answers must be written, not chosen from the first token, because routing only acts while writing.
- **Other models and machines.** We measured one model on one laptop. Rafid also runs Qwen3.5-122B-A10B and Qwen3.6-35B-A3B, which have not been evaluated here.
- **A direct comparison with cache-prior routing.** The bias rule of Skliar et al. [4] could be implemented in the same store and compared at matched hit rates.

Threats to validity. The reference answers for 8 of the 9 conversations come from a run before the thinking-tag fix, and the report conversation from a run after it. Both used exact routing and the same model, and no answer in those 8 conversations opened a thinking block. The checks are automatic: "any mention of the number" checks can accept a right number reached for the wrong reason. Each turn's example answer allows a manual review.

7 Conclusion

A 397B-parameter MoE model runs on a 31 GB Windows laptop. With exact routing it writes 5.5 tokens/s over 512 tokens, 1.5× llama.cpp's memory-mapped baseline, and reads prompts 6–7× faster. It writes 6.5 tokens/s, 1.75×, when the model may swap a low-ranked missing expert for a cached one. That swap changes about one token choice in a hundred and no

measured answer quality. Token-level fidelity, measured inside real conversations, is what makes the cost of approximate routing visible, where answer counts are blind to it.

Code and data availability

Rafid is released under the MIT licence: source, a Windows binary build, the Arabic conversation suite, an English template suite, and the raw result files of every table (`docs/results`, with `docs/REPRODUCE.md`). `tools/summarize_results.py` rebuilds every table from those files. Evaluators can import their own suites; each run records the SHA-256 of the suite file it used. Repository: `https://github.com/zaidlatif2010/rafid`.

References

[1] K. Alizadeh et al. LLM in a flash: Efficient large language model inference with limited memory. arXiv:2312.11514, 2023.

[2] A. Eliseev and D. Mazur. Fast inference of mixture-of-experts language models with offloading. arXiv:2312.17238, 2023.

[3] P. Tang et al. HOBBIT: A mixed precision expert offloading system for fast MoE inference. arXiv:2411.01433, 2024.

[4] A. Skliar et al. Mixture of cache-conditional experts for efficient mobile device inference. arXiv:2412.00099, 2024.

[5] D. Woods. Flash-MoE. As reported in Simon Willison's weblog, `https://simonwillison.net/b/9337`, March 2026; code at `https://github.com/fiveangle/flash-moe` (frozen fork).

[6] G. Gerganov and contributors. llama.cpp. `https://github.com/ggml-org/llama.cpp`, builds b12167 and b12169.